

필터와 리스너

학습 체크리스트

- ☐ HTTP 요청 및 응답의 길목에서 가로채는 서블릿 필터의 역할 이해
- ☐ doFilter()와 필터 체인(Filter Chain)의 요청/응답 제어 흐름 분석
- ☐ 서블릿 리스너(Listener)를 활용한 주요 생명주기 이벤트 감지 및 처리 습득
- ☐ 공통 관심사 처리 기술(필터 vs 인터셉터 vs AOP)의 동작 범위별 차이점 비교

서블릿 필터 개요

필터 (Filter)

웹 애플리케이션으로 들어오는 HTTP 요청과 서버가 내보내는 응답의 중간 길목에서 데이터를 가로채어 공통 전처리 또는 후처리 작업을 일괄 수행하는 객체

- 공통 처리의 이점:
 - 각 비즈니스 컨트롤러마다 중복으로 발생하는 횡단 관심사 로직을 한곳에서 일괄 통제함
 - 대표적으로 로그인 여부 확인, 한글 인코딩 설정, CORS 정책 설정, 보안 필터링 등을 수행함
- 서블릿 규격 준수:
 - 서블릿 컨테이너(Tomcat) 레벨에서 관리되며, 최종 뷰 템플릿이나 컨트롤러 진입 전에 수행됨

필터 체인과 동작 원리

- 필터 체인 (Filter Chain):
 - 서블릿 컨테이너는 등록된 필터들을 사슬처럼 순차적으로 연결하여 차례대로 구동함
 - 클라이언트 요청 발생 시 필터 체인을 타고 최종 목적지인 서블릿(Controller)으로 향함
- 체인 실행 흐름의 위임:
 - 각 필터 내부에서 `chain.doFilter(request, response)` 를 호출하여 다음 순서로 제어권을 넘겨줌
 - 만약 특정 검증에 실패하여 `chain.doFilter()` 호출을 건너뛰면, 해당 요청 흐름은 즉시 중단 및 차단됨

필터의 생명주기

- 서블릿 컨테이너 관리:
 - 서블릿과 마찬가지로 필터 역시 톰캣 등의 컨테이너가 라이프사이클 전체를 통제함
- `init(FilterConfig)` - 초기화:
 - 웹 애플리케이션 구동 시 또는 필터가 처음 로드될 때 **최초 1회** 구동되어 초기 설정 작업을 함
- `doFilter(req, res, chain)` - 요청 처리:
 - HTTP 요청이 유입될 때마다 독립적으로 매번 기동되어 전/후처리 로직을 제어함
- `destroy()` - 자원 해제:
 - 웹 애플리케이션이 종료되거나 컨테이너가 필터를 소멸시킬 때 **최초 1회** 구동되어 자원을 정리함

로그인 인증 체크 필터 선언

```
1  @WebFilter(urlPatterns = {"/dashboard/*", "/admin/*"})
2  public class LoginCheckFilter implements Filter {
3      public void init(FilterConfig config) throws ServletException {}
4      public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain)
5          throws IOException, ServletException {
6          // 인증 분기 및 위임 구현...
7      }
8      public void destroy() {}
9  }
```

로그인 검증 및 흐름 제어

```
1  HttpServletRequest httpRequest = (HttpServletRequest) request;
2  HttpServletResponse httpResponse = (HttpServletResponse) response;
3  HttpSession session = httpRequest.getSession(false); // 기존 세션 조회
4
5  boolean isLoggedIn = (session != null && session.getAttribute("loginUser") != null);
6  if (isLoggedIn) {
7      chain.doFilter(request, response); // 다음 필터 또는 서블릿 위임
8  } else {
9      httpResponse.sendRedirect(httpRequest.getContextPath() + "/login"); // 로그인창 이동
10 }
```

Spring Security와 보안 필터 체인

- 스프링 시큐리티의 아키텍처 근간:
 - 스프링 시큐리티 프레임워크 역시 서블릿 필터 기술을 바탕으로 모든 보안 메커니즘을 구동함
- 위임 프록시 패턴:
 - `DelegatingFilterProxy` : 서블릿 컨테이너 단에서 요청을 수집해 스프링 IoC 영역으로 넘겨주는 다리 역할 서블릿 필터
 - `FilterChainProxy` : 스프링 빈 컨테이너 내부에서 시큐리티 전용 보안 필터 목록(Security Filter Chain)들을 관리하고 순차 실행하는 제어 주체

서블릿 리스너 개요

리스너 (Listener)

웹 애플리케이션의 라이프사이클 주기 동안 발생하는 주요 상태 변화를 감지하고, 이벤트 발생 시 정의된 로직을 자동으로 구동하는 관찰자 객체

- 옵저버 패턴 (Observer Pattern):
 - 특정 객체의 상태 변화를 관찰(리스닝)하다가 이벤트 알림이 올 때 즉시 수신하여 처리하는 디자인 패턴
- 공통 초기화 및 자원 청소:
 - 웹 시작 시 필요한 글로벌 변수 로딩, 접속자 카운팅, 특정 스코프 변수 갱신 등을 대행함

리스너의 유형과 용도

- **ServletContextListener (웹 애플리케이션 라이프사이클 감지):**
 - 웹의 최초 구동 및 완전 종료 시점을 제어함
 - DB 커넥션 풀 초기화, 전역 설정 파일 읽기 등 무거운 연산을 전역적으로 대행함
- **HttpSessionListener (세션 상태 변화 감지):**
 - 사용자가 접속하여 세션이 생성되거나, 로그아웃/타임아웃으로 세션이 소멸하는 시점을 가로챌
 - 동시 로그인 제어, 현재 사이트 실시간 접속자 수 통계 집계 등에 활용됨
- **ServletRequestListener (HTTP 요청 유입 감지):**
 - 매번 HTTP 요청이 수집되어 스레드가 돌 때마다 생성을 감지하여 디버깅 로깅 등에 활용함

애플리케이션 시작/종료 이벤트 감지 구현

```
1  @WebListener
2  public class AppLifecycleListener implements ServletContextListener {
3      public void contextInitialized(ServletContextEvent sce) {
4          System.out.println("[시스템] 웹 애플리케이션이 시작되었습니다.");
5      }
6      public void contextDestroyed(ServletContextEvent sce) {
7          System.out.println("[시스템] 웹 애플리케이션이 안전하게 종료되었습니다.");
8      }
9  }
```

공통 관심사 처리 기술의 비교 (1/2)

비교 항목	서블릿 필터 (Filter)	스프링 인터셉터 (Interceptor)	스프링 AOP (Aspect)
동작 위치	서블릿 컨테이너 (Tomcat) 웹 입구 (DispatcherServlet 이전)	스프링 웹 영역 (DispatcherServlet과 컨트롤러 사이)	스프링 컨테이너 전체 영역 (서비스 및 영속성 메서드 레벨)
제어 대상	HTTP 요청/응답 객체 직접 제어 요청/응답 스트림 변환 및 차단	HTTP 요청/응답 정보 접근 가능 스프링 컨트롤러 뷰 모델 제어	메서드 매개변수 및 반환값 제어 웹 요청 정보 접근은 간접적

공통 관심사 처리 기술의 비교 (2/2)

비교 항목	서블릿 필터 (Filter)	스프링 인터셉터 (Interceptor)	스프링 AOP (Aspect)
주요 용도	공통 인코딩 설정 CORS 정책 적용 XSS 공격 사전 방어 등	로그인 정밀 권한 검증 컨트롤러 요청 처리 시간 측정 다국어 번역 매핑 처리 등	선언적 트랜잭션 (<code>@Transactional</code>) 관리 비즈니스 레이어 메서드 성능 감사 등
핵심 강점	웹 프레임워크와 무관하게 동작함	스프링 컨텍스트 내의 빈들을 주입받아 유기적으로 가공하기 쉬움	컨트롤러 외에 비즈니스 및 인프라 전체 스택에 유연하게 횡단 관심사 이식 가능

핵심 요약: 필터와 리스너

- 서블릿 필터 (Filter):
 - 요청/응답 길목에서 가로채는 횡단 관심사 공통 전/후처리기
 - `chain.doFilter()` 미호출 시 다음 요청이 차단되므로 흐름 위임 주의
- 서블릿 리스너 (Listener):
 - 이벤트 기반 옴저버 패턴을 통해 상태 변화를 추적하는 전역 관찰자
 - `ServletContextListener` 는 DB 풀 초기화 등 무거운 웹 시작 작업 전담
- 기술 포지셔닝:
 - 웹 전반 처리는 필터, 스프링 컨트롤러 단의 통제는 인터셉터, 핵심 비즈니스 메서드는 AOP를 적용함